# DLDCA Lab 4: Avengers Assemble

Week 5

August 28, 2026

## Introduction

In this lab we start writing x86-64 assembly. The tasks build up from the basic mechanics of registers and memory, through conditional branching, to working with arrays and loops, along with an additional graded task.

## Necessary tools

We shall be using `nasm` to assemble x86-64 assembly into object files, and `ld/gcc/g++` to link those object files into an executable. Before starting the lab, ensure both are installed:

```
$ nasm -v
NASM version [version]
$ ld -v # or gcc/g++ --version
GNU ld (GNU Binutils for Ubuntu) [version]
```

## Structure of submission/templates

Submission format:

```
your-roll-no/
|- labexam/
    |- single_num.s (TODO)
|- task0/
    |- add.s (TODO)
|- task1/
    |- overlap.s (TODO)
|- task2/
    |- dotprod.s (TODO)
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz` in lowercase. The command given below can be used to do the same

```
$ ./build_archive.sh
Enter Student Roll Number: your-roll-no
Packaging assignment files...
  [+] Copied: ...
Successfully created your-roll-no.tar.gz
```

# Tasks

## Lab Exam: Single Number

You are given an array of integers in which every value appears exactly twice, except for one value which appears only once. Find that value.

**Hint:** XOR has two properties that make this problem fall out almost for free: `x ^ x = 0` for any `x`, and XOR is commutative/associative, so the order you combine values in doesn't matter. Think about what the XOR of all numbers will be.

You can implement the pseudocode given below:

```
edi = 0

rcx = 0
.loopbegin:
    if rcx >= array_len: goto .loopend

    edi = edi XOR array[rcx*4]

    rcx = rcx + 1
    goto .loopbegin
.loopend:

exit(edi)
```

To run your code:

```
$ make labexam
...
Return Code: 42
```

## Task 0: Add Two Numbers

A warm-up to get comfortable with the basic mechanics of x86-64 assembly. Two numbers, `a` and `b`, are declared in `.data` as 64-bit values. Your job is to move `a` into `rdi`, add `b` to it, and exit the process with the result as the exit code (`rax = 60` is the `exit` syscall number; `rdi` holds the exit code).

Fill in the body of `_start` in `add.s`. You should only need two instructions.

To run your code:

```
$ make task0
...
Return Code: 30
```

You can also check the exit code directly after running the executable yourself with `echo $?`.

### Task 1: Do These Rectangles Overlap?

You are given two axis-aligned rectangles, `rect_a` and `rect_b`, each declared in `.data` as a small struct of four 64-bit fields:

```
rect_a:
    .x1: dq 0
    .y1: dq 0
    .x2: dq 10
    .y2: dq 10
```

NASM lets you address these fields directly by name from anywhere in the file, `[rect_a.x1]`, `[rect_b.y2]` etc. because any label starting with a '`.`' is treated as *local* to the most recent non-local label above it (here, `rect_a` or `rect_b`).

Your job: determine whether `rect_a` and `rect_b` overlap, and put `1` in `rdi` if they do, `0` otherwise, then exit with that value as the exit code.

**Definition of overlap.** Treat each rectangle's $x$-extent and $y$-extent as *open* intervals $(x_1, x_2)$ and $(y_1, y_2)$ – a rectangle does not include its own boundary. Two rectangles overlap if and only if both their $x$-intervals and $y$-intervals intersect as open intervals:

$$a.x_1 < b.x_2 \quad \text{AND} \quad b.x_1 < a.x_2 \quad \text{AND} \quad a.y_1 < b.y_2 \quad \text{AND} \quad b.y_1 < a.y_2$$

In particular: rectangles that only touch along a shared edge, with no interior overlap, do *not* count as overlapping.

Fill in `_start` in `overlaps.s`.

To run your code:

```
$ make task1
...
Return Code: 1
```

Feel free to edit the values in the structs to test your code.

### Task 2: Dot Product

You are given two vectors, `vec1` and `vec2`, each declared in `.data` as some number of 64-bit signed integers, along with `vecsize` (their length, computed for you from the difference between each vector's start and its `.end` label). With the values provided, their dot product

$$\texttt{vec1}[0] \cdot \texttt{vec2}[0] + \texttt{vec1}[1] \cdot \texttt{vec2}[1] + \cdots + \texttt{vec1}[n-1] \cdot \texttt{vec2}[n-1]$$

should come out to `9`.

This is your first real loop in this lab. Write a loop that computes the dot product and leaves the result in `rdi`, which is used directly as the exit code once the loop finishes.

**A note on register widths:** the vectors here hold 64-bit values (`dq`), so make sure the size of each memory access you use to read an element actually matches – reading the wrong width off the end of an array can silently pull in bytes from whatever happens to sit right after it in memory, without necessarily crashing or looking obviously wrong.

Fill in the body of `_start` in `dotprod.s`

To run your code:

```
$ make task2
...
Return Code: 9
```

Table 1: Some basic x86-64 Assembly Instructions

| Instruction | Operands | Effect |
|---|---|---|
| `mov` | `reg, reg/*mem/imm` | `reg` ← `reg`/`*mem`/`imm` |
| `mov` | `*mem, reg` | `*mem` ← `reg` |
| `lea` | `reg, *mem` | `reg` ← address of `*mem` |
| `add` | `reg, reg/*mem/imm` | `reg` ← `reg` + `reg/*mem/imm` |
| `add` | `*mem, reg/imm` | `*mem` ← `*mem` + `reg/imm` |
| `sub` | `reg, reg/*mem/imm` | `reg` ← `reg` − `reg/*mem/imm` |
| `sub` | `*mem, reg/imm` | `*mem` ← `*mem` − `reg/imm` |
| `inc` | `reg/*mem` | `reg/*mem` ← `reg/*mem` + 1 |
| `dec` | `reg/*mem` | `reg/*mem` ← `reg/*mem` − 1 |
| `neg` | `reg/*mem` | `reg/*mem` ← −`reg/*mem` (Two's complement) |
| `imul` | `reg, reg/*mem/imm` | `reg` ← `reg` × `reg/*mem/imm` |
| `imul` | `*mem, reg/imm` | `*mem` ← `*mem` × `reg/imm` |
| `div` | `reg` | `rax` ← `(rdx:rax)`/`reg`<br>`rdx` ← `(rdx:rax)`%`reg` |
| `cmp` | `reg/*mem, reg/imm` | Sets flags based on subtraction: `reg/*mem` - `reg/imm`.<br>ZF = 1 if equal, SF = 1 if result negative,<br>CF = 1 if borrow (unsigned), OF = 1 if signed overflow. |
| `and` | `reg, reg/*mem/imm` | `reg` ← `reg` ∧ `reg/*mem/imm` |
| `and` | `*mem, reg/imm` | `*mem` ← `*mem` ∧ `reg/imm` |
| `or` | `reg, reg/*mem/imm` | `reg` ← `reg` ∨ `reg/*mem/imm` |
| `or` | `*mem, reg/imm` | `*mem` ← `*mem` ∨ `reg/imm` |
| `xor` | `reg, reg/*mem/imm` | `reg` ← `reg` ⊕ `reg/*mem/imm` |
| `xor` | `*mem, reg/imm` | `*mem` ← `*mem` ⊕ `reg/imm` |
| `test` | `reg/*mem, reg/imm` | Sets flags based on bitwise AND: `reg/*mem` ∧ `reg/imm`.<br>ZF = 1 if result is zero, SF = 1 if high bit set,<br>CF and OF are cleared. No result is stored. |
| `jmp` | `label` | Unconditional jump to label |
| `je / jz` | `label` | Jump if Zero Flag (ZF) = 1 |
| `jne / jnz` | `label` | Jump if Zero Flag (ZF) = 0 |
| `jl` | `label` | Jump if Less (SF ≠ OF) |
| `jle` | `label` | Jump if Less or Equal (ZF = 1 or SF ≠ OF) |
| `jg` | `label` | Jump if Greater (ZF = 0 and SF = OF) |
| `jge` | `label` | Jump if Greater or Equal (SF = OF) |
| `syscall` | | Performs a syscall with syscall number stored in `rax`<br>Arguments passed in other registers starting from `rdi` |
| `push` | `reg` | Pushes the value in `reg` onto the stack |
| `pop` | `reg` | Pops the value on top of the stack onto `reg` |
| `call` | `label` | Calls the function called `label` |
| `ret` | | Pops the return address from stack and jumps to it |